Software Requirements Specification

for

Portfolio Management

Version 1.0 approved

Prepared by Devraj Sundrani, Tanmay Vishwakarma

Sardar Patel Institute Of Technology

July 27,2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History 1
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 2
2. Overall Description 3
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 4
2.7 Assumptions and Dependencies 5
3. External Interface Requirements 5
3.1 User Interfaces 5
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 6
4. System Features 7
4.1 Portfolio Creation and Management 7
4.2 Transaction Recording 8
4.3 Performance Analytics 9
4.4 Risk Assessment 9
4.5 Report Generation 10
5. Other Nonfunctional Requirements 11
5.1 Performance Requirements 11
5.2 Safety Requirements 11
5.3 Security Requirements 12
5.4 Software Quality Attributes 12
5.5 Business Rules 12
6. Other Requirements 13
Appendix A: Glossary 14
Appendix B: Analysis Models 14
Appendix C: To Be Determined List 15

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

*This document specifies the software requirements for the **Software Engineering (SE) Portfolio Management System (SEPMS)**, Release 1.0. The purpose of this SRS is to provide a comprehensive architectural and functional overview of the system for developers and project managers. This document covers the entire scope of the SEPMS, specifically detailing the requirements for the five core modules: Portfolio Creation and Management, Transaction Recording, Performance Analytics, Risk Assessment, and Report Generation.*

1.2 Document Conventions

This SRS follows standard IEEE documentation guidelines for requirements specification. The following conventions are used:

Bold text is used to highlight key terms, system actors, and critical constraints.

Italicized text is used for referencing external documents or emphasizing specific business logic.

Requirements are assigned a priority level using the MoSCoW method (Must have, Should have, Could have, Won't have). Priority levels assigned to high-level features are automatically inherited by their detailed sub-requirements unless explicitly stated otherwise.

Requirement IDs follow the format REQ-[Category]-[Number] (e.g., REQ-PORT-01).

1.3 Intended Audience and Reading Suggestions

The primary audience for this SRS includes:

- ***Project Managers / Portfolio Managers:** To understand the scope of portfolio management, milestone tracking (transactions), analytics, and risk assessments. Project managers should focus primarily on the Product Scope and the upcoming Specific Requirements sections for business logic.*
- ***Developers and Software Architects:** To understand the technical requirements, system constraints, security models, and data sources required to build the SEPMS. Developers should read the document sequentially, paying special attention to non-functional requirements, data source specifications, and the open-access permissions model.*

Reading Sequence: *It is recommended to read the Introduction and Product Scope first to understand the system's goals. Following this, readers should proceed to the specific functional requirements broken down by the five core modules.*

1.4 Product Scope

*The **SE Portfolio Management System (SEPMS)** is a centralized platform designed to manage and monitor software engineering assets, projects, and resources. Its primary objective is to provide project managers and development teams with a transparent, data-driven view of all ongoing software engineering initiatives.*

The software will encompass:

1. ***Portfolio Creation and Management:*** *Defining and aggregating software engineering projects, code repositories, resource allocations, and software assets into cohesive portfolios.*
2. ***Transaction Recording:*** *Logging both financial transactions (budget consumption, vendor costs) and project milestones (sprint completions, major deployments, phase gates).*
3. ***Performance Analytics:*** *Pulling data from project management tools and code repositories to visualize team velocity, budget burn-down rates, and project delivery metrics.*
4. ***Risk Assessment:*** *Utilizing historical project data and current bottlenecks to identify technical debt, resource constraints, and budget overrun risks.*
5. ***Report Generation:*** *Creating automated, exportable summaries of portfolio health.*

Business Alignment: *The SEPMS aligns with the corporate strategy of total transparency. To support this, a key system constraint is that **everyone** within the organization will have open access to view and interact with the data; there are no siloed user roles restricting visibility.*

1.5 References

- ***A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Seventh Edition****, Project Management Institute (PMI). (Non-Technical: Serves as the foundational framework for defining portfolio management structures, project milestones, and risk assessment methodologies used in this system).*
- ***IEEE Std 830-1998****, IEEE Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers. (Technical/Process: Provides the structural and formatting guidelines for this SRS document).*
- ***NIST Cybersecurity Framework (CSF) 2.0****, National Institute of Standards and Technology. Available at: https://www.nist.gov/cyberframework. (Non-Technical: Guides the non-functional security requirements, compliance constraints, and risk evaluation criteria for the Risk Assessment module).*
- ***Google Material Design 3 Guidelines****, Google. Available at: https://m3.material.io/. (Non-Technical/Design: Provides the overarching UI/UX principles, data visualization patterns, and accessibility standards for the system's dashboards and Report Generation modules).*
- ***Jira Software Cloud REST API Reference****, Atlassian. Available at: https://developer.atlassian.com/cloud/jira/software/rest/. (Technical: Crucial for backend*

integration to track project milestones, sprint velocity, and issue resolution for Transaction Recording and Performance Analytics).

- ***GitHub REST API Documentation**, GitHub Docs. Available at: https://docs.github.com/en/rest. (Technical: Required for external data ingestion regarding commit history, deployment frequency, and code churn to feed the Performance Analytics engine).*

2. Overall Description

2.1 Product Perspective

The SE Portfolio Management System (SEPMS) is a new, self-contained web-based application designed to replace fragmented, siloed project tracking spreadsheets and disparate departmental dashboards. While it acts as a standalone centralized hub, it is highly integrated into the organization's existing software engineering ecosystem.

SEPMS acts as an aggregation and orchestration layer. It interfaces directly with:

- ***Version Control and CI/CD Systems** (e.g., GitHub, GitLab, Jenkins) to pull software asset updates and deployment frequencies.*
- ***Project Management Tools** (e.g., Jira, Azure DevOps) to track agile metrics, milestones, and resource allocation.*
- ***Financial/ERP Systems** to record and pull budget data, vendor costs, and transaction histories.*

2.2 Product Functions

The system provides five primary functional pillars. (Detailed functional requirements for these are expanded in Section 3).

1. ***Portfolio Creation and Management:** Allows the aggregation of disparate software engineering elements—including active development projects, legacy software assets, repository links, and allocated human resources—into unified, trackable "Portfolios."*
2. ***Transaction Recording:** Facilitates the logging of both financial events (e.g., cloud infrastructure billing, software licensing costs, budget allocations) and project milestones (e.g., phase-gate approvals, sprint completions, major version releases).*
3. ***Performance Analytics:** Ingests data from connected external tools to calculate and visualize KPIs such as team velocity, budget burn-down, issue resolution time, and overall portfolio health.*
4. ***Risk Assessment:** Analyzes incoming data to flag technical debt accumulation, resource overallocation, timeline slippage, and budget overruns, outputting calculated risk scores.*
5. ***Report Generation:** Provides automated, customizable, and exportable reports (PDF, CSV, interactive dashboards) summarizing portfolio statuses for stakeholder review.*

2.3 User Classes and Characteristics

As dictated by the organization's radical transparency policy, ***the system employs a universal open-access model****. There are no siloed user roles restricting the visibility of data;* ***everyone*** *in the organization, from junior developers to executives, has full access to view portfolios, analytics, financials, and risk reports.*

While read-access is universal, users generally fall into the following personas based on how they use the system:

- ***Project / Portfolio Managers:*** *High-frequency users who actively create portfolios, define milestones, input financial transactions, and generate formal reports. They require intuitive interfaces for data entry and portfolio configuration.*
- ***Software Developers / Engineers:*** *Moderate-frequency users who utilize the system to view how their specific projects fit into the broader portfolio, check performance analytics, and understand identified risks (like technical debt).*
- ***Executive Leadership:*** *Low-frequency users who rely primarily on the Performance Analytics dashboards and automated Report Generation to make strategic funding and resource allocation decisions.*

2.4 Operating Environment

- ***Client-Side:*** *The application will be a web-based portal accessible via modern, standard desktop web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari).*
- ***Server-Side:*** *The system will be deployed on a scalable cloud infrastructure (e.g., AWS or Azure) utilizing containerized microservices (Docker/Kubernetes) to ensure high availability.*
- ***Database:*** *A relational database (e.g., PostgreSQL) will be used for transactional integrity (financials, milestones), supplemented by a time-series or NoSQL database for handling high-volume analytics and risk data.*

2.5 Design and Implementation Constraints

The development and architecture of the SEPMS are constrained by the following factors:

- ***Technology Stack Constraint:*** *The frontend must be developed using React.js to align with existing internal UI libraries. The backend API must be built using Node.js or Python to ensure maintainability by the current internal engineering team.*
- ***Timeline Constraint:*** *A Minimum Viable Product (MVP) encompassing Portfolio Management, Milestone Transactions, and basic Reporting must be delivered within* ***six (6) months*** *of project kickoff.*
- ***Budget Constraint:*** *Initial development is restricted to a fixed internal operational budget. Consequently, the architecture must favor open-source libraries and frameworks over paid commercial off-the-shelf (COTS) plugins wherever feasible.*
- ***Security and Compliance:*** *Even with the universal open-access policy for viewing data, the system must enforce strict data sanitization and prevent unauthorized data modification (e.g., altering financial transactions) using JWT-based authentication and audit logging.*

- ***Performance:*** *The system must render main portfolio dashboards within 3 seconds, assuming a load of up to 500 concurrent users.*

2.6 User Documentation

- ***Inline Tooltips and Walkthroughs:*** *Context-sensitive help elements integrated directly into the UI (e.g., explaining how a specific Risk Score is calculated).*
- ***User Manual / Wiki:*** *A comprehensive digital guide hosted on the internal corporate wiki detailing how to construct a portfolio, integrate external tools, and configure report generation.*
- ***API Documentation:*** *Swagger/OpenAPI documentation for developers outlining how to programmatically send transactions or pull analytics from the SEPMS.*

2.7 Assumptions and Dependencies

- ***Dependency on External APIs:*** *The Performance Analytics and Risk Assessment modules heavily depend on the availability, stability, and rate limits of external APIs (e.g., Jira, GitHub). If these systems experience downtime, SEPMS dashboard data may become temporarily stale.*
- ***Assumption of Standardized Data:*** *It is assumed that software engineering teams across the organization utilize standardized tagging, story point estimations, and milestone naming conventions in their respective project management tools. Without this, cross-portfolio performance analytics will be skewed or require complex data normalization mapping.*
- ***Infrastructure Availability:*** *It is assumed that the corporate DevOps team will provide the necessary cloud infrastructure and CI/CD pipelines to support the 6-month MVP timeline constraint.*

3. External Interface Requirements

3.1 User Interfaces

The user interface (UI) for the SEPMS will be a web-based, responsive application built using React.js. It must adhere to the organization's existing internal UI component library and style guide to ensure consistency with other internal tools.

Logical characteristics of the UI include:

- ***Global Navigation Structure:*** *A persistent left-hand sidebar or top navigation bar containing links to the core modules: Dashboard (Home), Portfolios, Transactions, Analytics, Risks, and Reports.*
- ***Open Access Paradigm:*** *Because all users have universal read access, there will be no hidden pages or "Access Denied" screens for viewing data. However, buttons for data mutation (e.g., "Add Transaction," "Edit Portfolio") will only be enabled for authenticated users with write permissions; otherwise, they will be visibly disabled (greyed out) with a tooltip explaining the required permission.*

- ***Data Visualization:*** *The Performance Analytics and Risk Assessment dashboards will heavily utilize interactive graphical elements (e.g., bar charts, line graphs for burn-down, heatmaps for risk) implemented via a standard charting library (e.g., Chart.js or D3.js).*
- ***Error Handling:*** *The system will use non-intrusive "toast" notifications in the upper right corner to display success (green), warning (yellow), and error (red) messages. Standard HTTP errors (e.g., 404, 500) will be handled with user-friendly error pages.*
- ***Responsive Design:*** *The UI must scale gracefully down to standard tablet resolutions (minimum 768px width) for executive review on mobile devices, though the primary target is desktop monitors.*

3.2 Hardware Interfaces

As a cloud-hosted, web-based application, the SEPMS does not have direct hardware interface requirements.

- ***Client Hardware:*** *The application will interact with the user's standard hardware (display, keyboard, mouse/trackpad) via the web browser.*
- ***Server Hardware:*** *Backend infrastructure will be entirely abstracted by the cloud service provider (e.g., AWS, Azure). There are no specific physical device integrations (such as scanners or IoT devices) required for this system.*

3.3 Software Interfaces

The SEPMS acts as an aggregation hub and must communicate with several external and internal software systems.

- ***Project Management Tools (e.g., Jira, Azure DevOps):***
  - *Purpose: To ingest project milestones, sprint data, issue statuses, and resource allocations.*
  - *Interface: RESTful APIs provided by the respective tools.*
  - *Data Exchanged: Inbound JSON payloads containing Epic/Task metadata, story points, status changes, and assignee data.*
- ***Version Control & CI/CD Systems (e.g., GitHub, GitLab):***
  - *Purpose: To monitor software asset health, deployment frequencies, and code activity.*
  - *Interface: Webhooks and REST/GraphQL APIs.*
  - *Data Exchanged: Inbound webhook events on pull requests, commits, and pipeline build statuses.*
- ***Financial / ERP Systems:***
  - *Purpose: To record and sync budget allocations, vendor costs, and infrastructural billing.*
  - *Interface: Secure RESTful API or scheduled flat-file (CSV) ingestion via an internal SFTP server, depending on the legacy status of the ERP.*
  - *Data Exchanged: Inbound cost data; outbound transaction reconciliation logs.*
- ***Corporate Identity Provider (IdP):***
  - *Purpose: To manage user authentication and enforce the "everyone has access" mandate seamlessly.*
  - *Interface: OAuth 2.0 / SAML 2.0 integration with the corporate IdP (e.g., Microsoft Entra ID or Okta).*
- ***Database Systems:***

- *Purpose: Data persistence.*
- *Interface: TCP/IP connections using standard database drivers (e.g., `pg` for PostgreSQL).*

3.4 Communications Interfaces

- ***Network Protocols:*** *All client-to-server and server-to-server communications must occur over* ***HTTPS (TLS 1.2 or higher)*** *to ensure data in transit is encrypted.*
- ***Backend-Frontend Communication:*** *The React UI will communicate with the Node.js/Python backend exclusively via a* ***RESTful API*** *using standard HTTP methods (GET, POST, PUT, DELETE) and exchanging JSON formatted payloads.*
- ***Automated Email Delivery:*** *The system will interface with a corporate SMTP server or third-party email service (e.g., SendGrid) to dispatch automated reports and real-time risk alerts. Emails will be formatted in HTML.*
- ***Synchronization Mechanisms:*** *External data from Jira and GitHub will be synchronized via scheduled CRON jobs (polling every 15-30 minutes) and event-driven webhooks to minimize API rate-limiting issues while keeping the portfolio dashboards adequately real-time.*

4. System Features

4.1 Portfolio Creation and Management

4.1.1 Description and Priority

Priority: High. This feature forms the foundation of the SEPMS. A "portfolio" in this context is defined as a logical grouping of Software Engineering (SE) assets. This includes active software development projects, legacy maintenance products, associated code repositories, cloud infrastructure environments, and assigned human resources.

4.1.2 Stimulus/Response Sequences

- *Action:* User selects "Create New Portfolio" and inputs fundamental metadata (Name, Description, Target Budget, Start/End Dates).
  - *Response:* System validates inputs, creates a unique Portfolio ID, and saves the record.
- *Action:* User links external SE assets (e.g., pastes a GitHub repo URL or selects a Jira Epic).
  - *Response:* System pings the external API to validate the asset exists, retrieves its metadata, and creates a relational link to the portfolio.
- *Action:* Any user attempts to view a portfolio.
  - *Response:* System grants immediate read access and displays the portfolio dashboard (enforcing the universal open-access policy).

4.1.3 Functional Requirements

- **REQ-PORT-01:** The system shall allow authorized users (Project Managers) to create, edit, and archive portfolios.
- **REQ-PORT-02:** A portfolio must support the linking of multiple, disparate SE assets, specifically: external source code repositories (via URI), project management epics/boards (via API ID), and external documentation links.
- **REQ-PORT-03:** The system shall allow the assignment of human resources (developers, QA, managers) to a portfolio, pulling user profiles from the corporate Identity Provider.
- **REQ-PORT-04:** The system shall enforce a universal "Read-Only" role for all authenticated users to view any portfolio's structure and metadata, while restricting "Write/Edit" actions to the explicitly assigned Portfolio Owners.
- **REQ-PORT-05:** The system shall prevent the deletion of a portfolio that contains active financial or milestone transactions; it must be "Archived" instead.

4.2 Transaction Recording

4.1.1 Description and Priority

Priority: High. Transactions in the SEPMS are distinct timestamped events that impact a portfolio. They are categorized into two types: **Financial Transactions** (e.g., budget allocations, AWS infrastructure billing, SaaS licensing costs) and **Project Milestones** (e.g., sprint completions, phase-gate sign-offs, critical bug resolutions, production deployments).

4.1.2 Stimulus/Response Sequences

- *Action:* User manually enters a financial expense via the "Add Transaction" UI form.
    - *Response:* System validates the data against the portfolio's active budget, records the transaction, and updates the total budget burn-down.
- *Action:* External CI/CD tool sends a webhook payload indicating a successful production deployment.
    - *Response:* System parses the payload, identifies the associated portfolio, and logs an automated "Milestone" transaction.

4.1.3 Functional Requirements

- **REQ-TXN-01:** The system shall provide an interface to manually log financial transactions, requiring fields for Amount, Currency, Category (e.g., Infrastructure, Vendor, Labor), Date, and Justification.

- **REQ-TXN-02:** The system shall provide an interface to manually log project milestones, requiring fields for Milestone Name, Target Date, Actual Date, and Status (Pending, Completed, Missed).
- **REQ-TXN-03:** The system shall expose a secure REST API endpoint to accept automated transaction creation (both financial and milestone) from external systems.
- **REQ-TXN-04:** The system shall maintain an immutable audit log for all recorded transactions. Transactions cannot be hard-deleted; erroneous entries must be offset by a reversing transaction to maintain data integrity.

4.3 Performance Analytics

4.3.1 Description and Priority

Priority: High. This module ingests raw data from internal transactions and external SE tools to calculate and visualize key performance indicators (KPIs) for development teams and project managers.

4.3.2 Stimulus/Response Sequences

- *Action:* User navigates to the Portfolio Analytics tab.
    - *Response:* System retrieves cached analytical data (refreshed every 15-30 minutes), calculates current KPIs, and renders interactive charts (e.g., line charts for budget, bar charts for velocity).

4.3.3 Functional Requirements

- **REQ-PERF-01:** The system shall calculate and visualize **Financial Performance**, specifically a Budget Burn-down chart comparing projected spending against actual logged financial transactions.
- **REQ-PERF-02:** The system shall calculate and visualize **Delivery Performance** by aggregating connected project management data, outputting metrics such as Team Velocity (story points completed per sprint) and Cycle Time (time from issue creation to resolution).
- **REQ-PERF-03:** The system shall calculate and visualize **Engineering Performance** by aggregating CI/CD and repository data, outputting metrics such as Deployment Frequency and Mean Time to Recovery (MTTR) for linked repositories.
- **REQ-PERF-04:** The system shall provide a unified cross-portfolio dashboard that aggregates top-level KPIs, allowing executives to compare the performance of multiple portfolios side-by-side.

4.4 Risk Assessment

4.4.1 Description and Priority

Priority: Medium-High. This module acts as an automated early-warning system. It processes the data from the Performance Analytics and Transaction modules through a rules engine to flag risks related to budget, schedule, and technical health.

4.4.2 Stimulus/Response Sequences

- *Action:* System's scheduled nightly CRON job executes the Risk Engine against all active portfolios.
    - *Response:* System calculates new risk scores, updates the database, and flags portfolios exceeding risk thresholds with visual indicators (e.g., red warning icons) on the main dashboard.
- *Action:* User clicks on a flagged Risk Indicator on a portfolio.
    - *Response:* System displays a detailed breakdown of the exact metrics that triggered the warning (e.g., "Budget is 20% over projection for Q3").

4.4.3 Functional Requirements

- **REQ-RISK-01:** The system shall calculate a **Financial Risk Score** by extrapolating current burn rates; if the projected final cost exceeds the portfolio budget by >10%, a warning is generated.
- **REQ-RISK-02:** The system shall calculate a **Schedule Risk Score** by comparing logged Milestone target dates against actual completion dates; consecutive missed milestones will trigger a schedule warning.
- **REQ-RISK-03:** The system shall calculate a **Technical Debt Risk Score** using proxy metrics derived from connected external tools (e.g., an increasing ratio of "Bug" tickets vs. "Feature" tickets over 3 consecutive sprints).
- **REQ-RISK-04:** The system shall aggregate all individual risk vectors into an overall Portfolio Health Score (e.g., Red, Yellow, Green) visible to all users.

4.5 Report Generation

4.5.1 Description and Priority

Priority: Medium. The system must allow users to extract portfolio data into standardized, shareable formats for offline review, executive presentations, or external auditing.

4.5.2 Stimulus/Response Sequences

- *Action:* User selects a date range, selects a portfolio, and clicks "Export to PDF."
    - *Response:* System dynamically compiles the requested data, generates the PDF, and initiates a file download to the user's local machine.
- *Action:* First day of the month is reached (automated trigger).
    - *Response:* System generates the Monthly Executive Summary report and emails it to the configured distribution list.

4.5.3 Functional Requirements

- **REQ-RPT-01:** The system shall support the generation of the following report types:
    - *Executive Health Summary* (High-level budget, major milestones, overall risk score).
    - *Financial Ledger* (Detailed list of all logged financial transactions).
    - *Milestone Status Report* (Detailed list of completed vs. pending project milestones).
- **REQ-RPT-02:** The system shall allow reports to be exported in universally accessible formats, specifically PDF (for visual sharing) and CSV (for raw data manipulation).
- **REQ-RPT-03:** The system shall provide a scheduling interface allowing users to configure automated, recurring report generation (e.g., weekly, monthly) delivered via email.
- **REQ-RPT-04:** Report generation parameters must allow users to filter data by specific Portfolios, Date Ranges, and Risk Severity Levels prior to generation.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

To ensure the system is adopted seamlessly by high-frequency users (Project Managers) and low-frequency users (Executives), it must remain highly responsive even under load.

- ***REQ-PERF-05:*** *The primary Portfolio Dashboard and Performance Analytics pages must fully render within* ***three (3) seconds*** *for 95% of requests under a standard load of up to 500 concurrent users.*
- ***REQ-PERF-06:*** *API response times for standard read operations (fetching portfolios, viewing transactions) shall not exceed* ***500 milliseconds****.*
- ***REQ-PERF-07:*** *The generation of complex, multi-page PDF reports must complete within* ***ten (10) seconds****. If the data size dictates a longer processing time, the system must offload the task to a background queue and notify the user via email upon completion.*
- ***REQ-PERF-08:*** *Background synchronization jobs (polling Jira, parsing GitHub webhooks) must operate asynchronously and must not degrade the front-end user experience during peak business hours.*

5.2 Safety Requirements

While the SEPMS is not a life-critical system, the financial and strategic data it handles is highly sensitive. "Safety" in this context refers to data integrity and the prevention of catastrophic business decisions based on erroneous system states.

- ***REQ-SAFE-01:*** *The system shall implement automated, daily database backups with point-in-time recovery (PITR) capabilities to prevent catastrophic data loss in the event of an infrastructure failure.*
- ***REQ-SAFE-02:*** *The Risk Assessment engine must fail safely; if an external data source (e.g., GitHub) goes offline and returns malformed data, the system must not calculate a false "Critical Risk" score. It should instead display an "Unknown/Data Unavailable" state.*
- ***REQ-SAFE-03:*** *The system must prevent concurrent edit collisions. If two users attempt to modify the same portfolio metadata simultaneously, the system will implement optimistic concurrency control to reject the second save attempt and prompt the user to refresh.*

5.3 Security Requirements

Despite the organizational policy of radical transparency, the system must rigorously secure how data is modified and transported.

- ***REQ-SEC-01: Authentication:*** *All users must authenticate via the corporate Identity Provider (IdP) using OAuth 2.0 / SAML 2.0. Anonymous access to any part of the system is strictly prohibited.*
- ***REQ-SEC-02: Authorization:*** *The system must enforce the Universal Read Access policy. However, all HTTP POST, PUT, and DELETE operations must be protected by Role-Based Access Control (RBAC) middleware verifying the user is the assigned "Portfolio Owner" or a "System Administrator."*
- ***REQ-SEC-03: Data in Transit and at Rest:*** *All network traffic must be encrypted using TLS 1.2 or higher. All persistent data in the database (specifically financial transactions) must be encrypted at rest using AES-256 standard encryption.*
- ***REQ-SEC-04: Auditability:*** *The system must maintain an immutable audit trail for all write operations. Every financial or milestone transaction logged, edited, or offset must be recorded alongside the timestamp, the authenticating User ID, and the IP address.*

5.4 Software Quality Attributes

- ***Usability:*** *Given that executives and external stakeholders will use the system infrequently, the Performance and Risk dashboards must be entirely intuitive, requiring zero formal training to interpret the visual data (e.g., standard red/yellow/green indicators).*
- ***Interoperability:*** *Because the SEPMS's value is derived from aggregating external data, the system architecture must be highly interoperable, utilizing standard RESTful APIs and webhook receivers to easily plug in new project management or CI/CD tools in the future.*
- ***Availability:*** *The system must guarantee a* ***99.9% uptime*** *during standard corporate business hours (Monday–Friday, 8:00 AM–6:00 PM local time). Planned maintenance and deployments must occur outside this window.*
- ***Maintainability:*** *The backend codebase (Node.js/Python) must adhere strictly to modular, microservice-inspired design patterns so the internal engineering team can easily maintain or extend specific modules (like the Risk Engine) without disrupting the entire application.*

5.5 Business Rules

These operational principles govern the business logic implemented within the system's functional requirements.

- ***BR-01: The Universal Access Rule.*** *No organizational portfolio, budget data, or performance metric may be hidden behind "private" access controls. If a portfolio exists in the system, any authenticated employee can view it.*
- ***BR-02: Immutability of Transactions.*** *Financial transactions and project milestones represent historical facts. Users are prohibited from hard-deleting transactions. To fix an error, a user must submit a reversing/offsetting transaction (e.g., logging a -$500 transaction to cancel an accidental +$500 entry) with an attached justification.*
- ***BR-03: Portfolio Integrity.*** *A portfolio cannot be deleted once a transaction (financial or milestone) has been attached to it. It can only be marked as "Archived" to preserve historical roll-up reporting.*
- ***BR-04: Automated Data Supremacy.*** *If a conflict occurs between a manually entered milestone status and an automated webhook trigger (e.g., a user marks a release "pending" but the CI/CD pipeline reports a successful production deployment), the system will always favor and record the automated system's data.*

6. Other Requirements

6.1 Database Requirements

- ***Relational Integrity:*** *The system requires an ACID-compliant relational database (e.g., PostgreSQL) to ensure that financial transactions and portfolio milestones are recorded with strict data integrity.*
- ***Time-Series Data:*** *To support the Performance Analytics and Risk Assessment modules, the database architecture must efficiently handle time-series data to track historical changes in velocity, risk scores, and burn-down rates over time.*
- ***Caching:*** *A caching layer (e.g., Redis) shall be utilized to store frequently accessed, compute-heavy Performance Analytics data to meet the 3-second dashboard rendering requirement, refreshing at minimum every 15-30 minutes.*

6.2 Internationalization and Localization Requirements

- ***Language:*** *As an internal corporate application, the initial release of the user interface and generated reports shall be strictly in English (en-US).*
- ***Multi-Currency Support:*** *Because software engineering resources and infrastructure may span global regions, the Transaction Recording module must support inputs in multiple major currencies (USD, EUR, GBP, INR) and automatically normalize them to a base corporate currency (e.g., USD) for aggregated Portfolio reporting.*

6.3 Legal and Data Retention

- ***Data Retention Policy:*** *To comply with internal corporate auditing standards, all financial and milestone transactions, along with their associated audit logs, must be retained for a minimum of seven (7) years.*
- ***Archival State:*** *Portfolios that are no longer active cannot be permanently deleted; they must be transitioned to an "Archived" state, moving their data to lower-cost cold storage while remaining accessible for historical reporting.*

Appendix A: Glossary

- ***API (Application Programming Interface):*** *A set of rules that allows different software entities to communicate. Used by SEPMS to pull data from Jira, GitHub, etc.*
- ***Burn-down Chart:*** *A visual representation of work left to do versus time, or budget remaining versus time.*
- ***CI/CD (Continuous Integration / Continuous Deployment):*** *The automated process of building, testing, and deploying code.*
- ***Epic:*** *A large body of work in agile project management that can be broken down into smaller tasks or user stories.*
- ***IdP (Identity Provider):*** *A system that manages identity information and provides authentication services (e.g., Okta, Microsoft Entra ID).*
- ***MTTR (Mean Time to Recovery):*** *A metric tracking the average time it takes to recover from a product or system failure.*
- ***Portfolio:*** *In SEPMS, a logical container aggregating related software engineering projects, code repositories, team members, and budgets.*
- ***RBAC (Role-Based Access Control):*** *A method of restricting network access based on the roles of individual users within an enterprise.*
- ***Transaction:*** *A discrete, timestamped historical event recorded in SEPMS. Can be financial (an expense) or project-based (a milestone).*
- ***Velocity:*** *An agile metric calculating the amount of work (usually in story points) a team successfully completes within a sprint.*

Appendix B: Analysis Models

B.1 Entity-Relationship (ER) Overview

The core database relationships center around the Portfolio entity:

- **Portfolio (1) -> (Many) Transactions:** A single portfolio has many financial and milestone transactions.
- **Portfolio (1) -> (Many) Assets:** A portfolio connects to multiple external assets (e.g., Jira Epics, GitHub Repositories).
- **Portfolio (1) -> (1) Risk Profile:** Each portfolio has a uniquely calculated daily risk record.
- **User (Many) <-> (Many) Portfolio:** Users are assigned to portfolios via a junction table defining their specific relationship (e.g., Owner, Contributor).

B.2 High-Level Data Flow Diagram (DFD)

1. **External Sources (Jira, GitHub, ERP) --> Data Ingestion Layer:** Data enters via scheduled polling or webhooks.
2. **Data Ingestion Layer --> Database & Analytics Engine:** Data is normalized and stored.
3. **Analytics Engine --> Risk Engine:** Nightly CRON jobs evaluate the normalized data against risk thresholds.
4. **Database & Analytics Engine --> Client UI & Report Generator:** Data is pushed to the React frontend for visualization or compiled into PDFs based on user requests.

Appendix C: To Be Determined List

The following items require further clarification or technical spike research before development can finalize:

- ***TBD-1: Report Generation Library:*** *The specific open-source Node.js/Python library to be used for dynamic PDF generation (e.g., Puppeteer, PDFKit) is pending a performance evaluation.*
- ***TBD-2: Risk Engine Algorithm Weights:*** *The exact mathematical weighting of individual risk vectors (e.g., Schedule vs. Financial vs. Technical Debt) used to calculate the final "Overall Risk Score" is pending final approval from the PMO (Project Management Office).*
- ***TBD-3: External API Rate Limits:*** *The maximum allowable polling frequency for Jira and GitHub needs to be confirmed with the internal IT department to avoid IP blacklisting or rate-limiting errors.*
- ***TBD-4: ERP System Integration Method:*** *It is yet to be determined whether the legacy internal ERP system will support real-time REST API integration for financial data or if it will require a nightly flat-file (CSV) SFTP batch drop.*